ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/353586348

A sequence labeling model for catchphrase identification from legal case documents

Article *in* Artificial Intelligence and Law · July 2021

DOI: 10.1007/s10506-021-09296-2

CITATIONS
21

READS
145

4 authors, including:



Arpan Mandal
Indian Institute of Technology Kharagpur
6 PUBLICATIONS **144** CITATIONS
SEE PROFILE



Kripabandhu Ghosh
Indian Statistical Institute
68 PUBLICATIONS **843** CITATIONS
SEE PROFILE



Saptarshi Ghosh
Indian Institute of Technology Kharagpur, India
162 PUBLICATIONS **3,154** CITATIONS
SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Social Network Analysis View project

Social Media for Emergency Relief and Preparedness View project

All content following this page was uploaded by Arpan Mandal on 13 October 2022.

The user has requested enhancement of the downloaded file.

Artificial Intelligence and Law
https://doi.org/10.1007/s10506-021-09296-2

ORIGINAL RESEARCH

A sequence labeling model for catchphrase identification from legal case documents

Arpan Mandal[1] · **Kripabandhu Ghosh**[2] · **Saptarshi Ghosh**[3] · **Sekhar Mandal**[1]

Accepted: 24 June 2021
© The Author(s), under exclusive licence to Springer Nature B.V. 2021

Abstract
In a Common Law system, legal practitioners need frequent access to prior case documents that discuss relevant legal issues. Case documents are generally very lengthy, containing complex sentence structures, and reading them fully is a strenuous task even for legal practitioners. Having a concise overview of these documents can relieve legal practitioners from the task of reading the complete case statements. Legal catchphrases are (multi-word) phrases that provide a concise overview of the contents of a case document, and automated generation of catchphrases is a challenging problem in legal analytics. In this paper, we propose a novel supervised neural sequence tagging model for the extraction of catchphrases from legal case documents. Specifically, we show that incorporating document-specific information along with a sequence tagging model can enhance the performance of catchphrase extraction. We perform experiments over a set of Indian Supreme Court case documents, for which the gold-standard catchphrases (annotated by legal practitioners) are obtained from a popular legal information system. The performance of our proposed method is compared with that of several existing supervised and unsupervised methods, and our proposed method is empirically shown to be superior to all baselines.

Keywords Legal catchphrases · Catchphrase extraction · Keyphrase extraction · Legal analytics · Sequence tagging · BiGRU · CRF · Doc2Vec

1 Introduction

Any country that follows a Common Law system (e.g., USA, UK, Canada, Australia, India, and many others) has two sources of law for arguing an ongoing case—(1) the laws promulgated by the legislature, and (2) the precedents that are prior cases in which the situation was similar to the ongoing case. In this system of law, the verdict

✉ Arpan Mandal
amarnamarpan@gmail.com

Extended author information available on the last page of the article

Published online: 30 July 2021

given in the precedents can be cited to claim a similar verdict in the ongoing case. Hence, legal practitioners need to go through many prior case documents to find relevant precedents that can be cited in the ongoing case. Case documents are generally lengthy (often spanning hundreds of pages) and contain complex sentences that prefer formal structure rather than readability. So, reading a case document fully is a strenuous and time-consuming task even for a legal practitioner. Hence, it is advantageous to have a concise overview of the contents of a case document.

Catchphrases are a set of short phrases that collectively provide a concise representation of a legal case document. Basically, these are multi-word phrases that give an idea about the context and content of a document (Galgani et al. 2012; Witten et al. 1999; Liu et al. 2009; Giamblanco and Siddavaatam 2017; Wu and Li 2008). These catchphrases are not only formed of legal domain-specific important terms. but also terms or phrases that have document-specific importance. Note that, although the concept of *keyphrases* is prevalent in the literature and has regained recent interest, we use the term *catchphrases* as this is preferred for describing keyphrases in the legal context (Galgani et al. 2012; Tran et al. 2018, 2020).

While describing the idea of *catchphrases*, it is important to mention two properties that distinguish catchphrases from other phrases. These properties are inspired by a recent survey paper on keyphrase extraction (Firoozeh et al. 2019). The properties are as follows:

- *Specificity* This property dictates that a catchphrase should be document-specific, i.e, the same phrase might be chosen as a catchphrase in one document and not in another if the context of the two documents are different.
- *Well-formedness* This aspect of catchphrases increases readability when presented to humans. This property prohibits forms of catchphrases that are stemmed such as "*Constitut*" instead of "*Constitution*" and also prohibits incomplete noun phrases such as "*Constitution of* " instead of "*Constitution of India*".

Based on our discussions with law experts (details below), these properties hold for catchphrases in legal documents.

Apart from providing an overview of documents to readers, catchphrases have other utilities as well, such as they can be used in legal information systems to build indexes of legal documents. Thus, all popular legal information systems such as *Thomson Reuters Westlaw*[1] and *Manupatra*[2] (a legal information system popular in India) provide catchphrases for case documents, which are generally assigned manually by legal practitioners (see Table 2 for examples of catchphrases). Since it is expensive to employ legal practitioners to manually identify catchphrases, automating the process of identifying catchphrases is a well-known challenge in legal analytics, and this is the challenge that is addressed in this work.

[1] https://legal.thomsonreuters.com/en/products/westlaw.
[2] https://www.manupatrafast.com/.

1.1 Types of catchphrases and extraction methods

Catchphrases can mainly be of two types—*abstractive* and *extractive*. For a given target document, there can be phrases that are relevant to its content, but are *not* present within the document text. Such catchphrases that may not be present within the document text are called *abstractive* catchphrases (analogous to an abstractive summary of a text). In contrast, *extractive* catchphrases are those that are extracted from the text of the document (analogous to an extractive summary). Prior works have developed automatic methods for identifying both *abstractive* catchphrases (Qazvinian et al. 2010; Caragea et al. 2014), as well as *extractive* catchphrases (Galgani et al. 2012; Mandal et al. 2017; Tran et al. 2018; Tomokiyo and Hurst 2003).

We intended to understand which type of catchphrases (extractive or abstractive) is preferred by legal practitioners. To this end, we discussed with several law practitioners from two reputed law schools in India—The West Bengal National University of Juridical Sciences (www.nujs.edu), and, Rajiv Gandhi School of Intellectual Property Law (www.iitkgp.ac.in/department/IP). From these discussions, it was understood that the *law practitioners prefer extractive catchphrases* that are present within the text of the document. Additionally, the law practitioners prefer that extracted catchphrases should have *exact verbatim match with the gold-standard catchphrases*, instead of just partial match (where it is sufficient to have only a few of the constituent words of an extracted phrase to match with gold-standard catchphrases). Hence, in this work, we focus on identifying extractive catchphrases that would have verbatim match with the gold-standard catchphrases identified by law practitioners.

It can be noted that some of the prior studies on legal catchphrase detection (Galgani et al. 2012; Tran et al. 2018) relied upon a particular dataset of case documents from the Federal Court of Australia along with their catchphrases (the dataset was introduced in (Galgani et al. 2012)). The catchphrases in this dataset consists of very long phrases with an average length of over eight (08) terms. A few examples of catchphrases from a specific document of this dataset are shown in Table 1. We find that in this dataset, 79% of the catchphrases do *not* have any verbatim string match with any part of the corresponding document text i.e, 79% of the catchphrases can not be extracted from the document text. Hence, this dataset is *not* appropriate for applying extractive catchphrase detection methods. Hence, we have developed our own dataset comprising of case documents of the Indian Supreme Court, and gold standard catchphrases provided by the Manupatra legal system (https://www.manupatrafast.com/) for these documents. This dataset has 94.9% of its catchphrases present within the text of the documents. Table 2 (second column) shows examples of catchphrases for this dataset (detailed in Sect. 3).

The available methods of catchphrase extraction can be broadly categorised in another way into two groups: *unsupervised* and *supervised*. Supervised methods use available examples (a training set) of catchphrases and the document text, to learn a model for extracting catchphrases from new documents. On the other hand, unsupervised methods e.g., (Mandal et al. 2017; Galgani et al. 2012) do not make use of any such training examples. Supervised methods generally perform better than unsupervised methods (as is shown later in the paper) and, hence we primarily focus on

Table 1 Examples of catchphrases of a sample document from the dataset by Galgani et al. (2012), showing catchphrases that are present in the document text and others that are not

Catchphrases present within the document text
rfa forestry operation
Commonwealth land
epbc act
*Catchphrases **not** present within the document text*
Application for review of two decisions made by minister under the environment protection and biodiversity conservation act 1999 (cth) (epbc act)
First decision designating proposed action as controlled action and specifying controlling provisions
Second decision that relevant impacts of controlled action be assessed on preliminary documentation
Whether valid referral under epbc act to support decisions
Whether in making first decision the minister failed to take into account relevant considerations or took into account an irrelevant consideration
s 75(2b) of epbc act
Whether minister misconstrued s 87(5) of epbc act in making second decision
Whether denial of procedural fairness in making of second decision
Whether second decision affected by apprehended bias
Whether second decision involved improper exercise of power
Whether second decision manifestly unreasonable
Administrative law

supervised catchphrase extraction in this paper. To our knowledge, there is only one prior work on supervised catchphrase extraction from legal case documents (Tran et al. 2018), which employs a neural model called DeepCNN that is trained on sets of labeled documents. The DeepCNN model is considered as a baseline in this work.

1.2 Research goals and contributions

The main goal of our research is to develop a supervised model for catchphrase extraction from legal court case documents. To this end, we hypothesize that—enhancing a sequence tagging model with document-specific information results in more accurate[3] extraction of catchphrases from legal documents. We propose a novel sequence labeling model for extracting catchphrases from legal documents, which is capable of utilising document-specific information (pre-trained document embeddings) and requires no hand-crafted features for its training. Our experiments over the dataset of Indian Supreme Court case documents confirm our hypothesis. We compare our proposed model with several baseline methods (Galgani et al. 2012; Witten et al. 1999; Tran et al. 2018; Mandal et al. 2017; Medelyan 2009) that

[3] Accuracy is a well-known set-based evaluation metric to measure the performance of classification algorithms, that measures what fraction of instances are correctly classified by a model. In the present context, accuracy can be used to measure what fraction of catchphrases are correctly identified by a method.

Table 2 Examples of catchphrases of three Indian Supreme Court case documents

Case ID	Gold standard catchphrases	Catchphrases by our proposed model
INSC.2016.881	accused, **armed forces**, arrest, **census**, **commission of inquiry**, **compensation**, **complete justice**, constitution of india, **discrimination**, educational institution, **election**, **fir**, **franchise**, **fundamental right**, human right, interim order, **mandamus**, **minorities**, national human rights commission, **recommendation**, **remedy**, retired judge, writ of mandamus	**commission of inquiry**, **discrimination**, **census**, **compensation**, **franchise**, **recommendation**, public interest, **minorities**, **remedy**, **fir**, **armed forces**, commission, **mandamus**, suppression, **fundamental right**, **complete justice**, **election**, retired
INSC.2015.732	adjudication, administrative power, back wage, **compensation**, **competent authority**, conditions of service, continuous service, discharge, **disciplinary action**, dismissal, general rule, industrial dispute, **presiding officer**, **previous sanction**, **reasonable opportunity**, reinstatement, **retrenchment**, settlement, social justice, vacancies	employee, parliament, employer, letters patent appeal, **reasonable opportunity**, **presiding officer**, **retrenchment**, **compensation**, **disciplinary action**, **previous sanction**, remedy, **competent authority**, vote, presiding
INSC.2016.1696	**affirmation**, **capital offence**, causing disappearance, **commission of the offence**, commit an offence, **conspiracy**, **contrary to law**, **corruptly making report**, criminal conspiracy, **culpability**, **delinquent**, **disappearance of evidence**, **false information**, giving false information, **imprisonment for life**, **judicial proceeding**, **maliciously**, medical expert, medical practitioner, **negligence**, **oath**, oath or affirmation, offence punishable with death, **public servant**, **punishable with death**, punishable with imprisonment, **reason to believe**, **rigorous imprisonment**, **screening an offender**, stage of a judicial proceeding, **suppression**	**disappearance of evidence**, **maliciously**, **false information**, **affirmation**, causing, **oath**, **commission of the offence**, **rigorous imprisonment**, **judicial proceeding**, **imprisonment for life**, medical negligence, **culpability**, **contrary to law**, **capital offence**, jurisdiction, **reason to believe**, criminal, **public servant**, public servant corruptly, **screening an offender**, **punishable with death**, **conspiracy**, **negligence**, **corruptly making report**, offence, punishable with death imprisonment for life, **suppression**, **delinquent**

The 2nd column shows the catchphrases from the Manupatra legal system, that are identified by law practitioners (considered the gold standard). The 3rd column shows the catchphrases identified by our proposed algorithm D2V-BiGRU-CRF. The matching catchphrases are marked in bold

include state-of-the-art supervised and unsupervised methods. The performance of the proposed model is *statistically significantly better than that of all baselines in all the evaluation measures* at 95% confidence level ($p < 0.05$) by Wilcoxon signed-rank test (Siegel 1956). Examples of catchphrases extracted by our proposed method are also shown in Table 2 (last column).

Another contribution of our research is to experiment with existing sequence tagging models upon the task of extracting catchphrases from legal court case documents. Sequence labeling has earlier been used in keyphrase extraction from scholarly articles (Alzaidy et al. 2019; Zhu et al. 2020) but, it has never been used for catchphrase extraction from legal documents. In this paper, we show how the problem of catchphrase extraction from legal documents can be modelled as a *sequence labelling* task. To our knowledge, this is a new form of modelling the task which enables us to apply state-of-the-art sequence labeling models to this problem. Specifically, we apply Conditional Random Fields (CRF) (Lafferty et al. 2001), and BiGRU-CRF (a variation of BiLSTM-CRF (Lample et al. 2016)) models to retrieve the legal catchphrases.

In summary, we have the following contributions:

- We model the task of legal catchphrase extraction as a sequence labeling task, which, to our knowledge, has not been tried earlier. Modeling the catchphrase extraction problem as a sequence labeling task opens the problem to be solved using any state-of-the-art sequence labelling method. We experiment with several sequence labeling models and check their performance for the task of catchphrase extraction from legal documents.
- We propose a novel catchphrase extraction model, that uses document embeddings to aid a sequence labeling model. Our proposed method shows a 163% improvement in performance (in terms of F1-Score) over the closest existing baseline DeepCNN (Tran et al. 2018). The implementation of the proposed model (D2V-BiGRU-CRF) is available at https://github.com/amarnamarpan/D2V-BiGRU-CRF.

The rest of the paper is organized as follows. Section 2 surveys existing works on legal catchphrase extraction. The dataset is presented in Sect. 3. Section 4 details the proposed catchphrase extraction methodology. The evaluation setup is explained in Sect. 6, and experimental results (including comparison of our proposed method with baselines) are presented in Sect. 7. Section 8 discusses the implications that can be derived from the experimental results, and finally summarizes our contributions and concludes the paper.

2 Related works

There exist several prior works on catchphrase extraction, not only on court case documents but also on other types of documents. This section surveys a few of these works.

2.1 Catchphrase extraction from legal case documents

Both unsupervised and supervised methods have been developed for catchphrase extraction from legal court case documents.

Unsupervised methods Galgani et al. (Galgani et al. 2012) described an unsupervised scoring method named *MyScore*, that can be used to score sentences or phrases based on their importance. Once all the phrases are scored, the top-scored phrases for a document can be chosen as its catchphrases. In the original paper, the authors used this method to score *sentences* and chose the top-scoring sentences as the resultant catchphrases. We have considered a variant of the *MyScore* algorithm as a baseline in this work.

One of our prior works (Mandal et al. 2017) developed an unsupervised catchphrase extraction method called *PSlegal*, that scores phrases on the basis of their legal-specific importance and document-specific importance. Specifically, PSlegal uses the difference in the probability distribution of the occurrence of a phrase across different domains to measure the domain-specific importance. *PSlegal* has also been used as a baseline in the present paper.

Supervised methods Tran et al. (Tran et al. 2018, 2020) developed supervised methods for catchphrase extraction from court case documents. They used a scoring function based on a Convolutional Neural Network model (called DeepCNN) to score words of a document. In (Tran et al. 2018), the scoring function was used to choose a predefined number of highly-scored words (called *anchor words*). Then, a predefined number of words neighboring to each anchor word are considered as a catchphrase. It can be noted, that the catchphrases extracted by this method always consists of a fixed number of words, and are not of variable lengths.

As mentioned in the Introduction, Galgani et al. (Galgani et al. 2012) introduced a collection of case statements from Australian Courts along with their catchphrases.[4] Tran et al. (Tran et al. 2018) also originally applied their DeepCNN model over the same dataset that was introduced by Galgani et al. (Galgani et al. 2012). As stated earlier, for any given document from this dataset, 79% of its catchphrases are *not* present in the document text on average. Hence, the evaluations in (Galgani et al. 2012) and (Tran et al. 2018) were based on ROUGE scores (popular metrics for text summarization) that rely on partial matches of catchphrases (Galgani et al. 2012). Since the objective of the present work is to identify extractive catchphrases that match verbatim with catchphrases identified by legal experts, we have considered slight variants of MyScore (Galgani et al. 2012) and DeepCNN (Tran et al. 2018) as our baselines.

2.2 Catchphrase extraction from other documents

There have been several works on catchphrase extraction from different types of documents. For instance, Frank et al. (Frank et al. 1999) showed how domain-specific

[4] The dataset is available online at https://archive.ics.uci.edu/ml/datasets/Legal+Case+Reports.

information can be exploited for better catchphrase extraction from documents of a particular domain. It is one of the first papers that used the classic two phase approach of firstly identifying candidate phrases out of document and secondly pruning out the actual catchphrases among these candidates. Liu et al. (Liu et al. 2010) uses a scoring method based on semantic word-graph to score the candidate-phrases of a document, and to finally select a few highly scored candidate-phrases as the final keyphrases. Zahoor et al. (Zahoor and Bajwa 2014) developed a NLP-based method for catchphrase extraction from software license agreements. It uses several phases of lexical, syntactical and morphological heuristics to directly identify impactful phrases out of the agreements. This method is very specific to license agreements and hence is not applicable to our problem.

In one of the recent works, Lossio-Ventura et al. aims to extract terminologies from biomedical documents (Lossio-Ventura et al. 2016). They use the traditional approach of first identifying candidate terms and hence using ranking methods to select the top-ranked ones as valid terminologies. A recent work by Shi et al. (Shi et al. 2017) uses knowledge-graph to score candidate-phrases out of documents. They specifically use DBpedia (Bizer et al. 2009) to form the knowledge-graph between entities and use Personalised PageRank (Haveliwala 2002) to score the entities in a document. Interestingly, this recent method also relies on the classic two-phase approach of first identifying the candidate phrases and hence scoring them using graph-based scores. Another recent work by Vega-Oliveros et al. attempts to extract single-word terms from a given text. The text is represented as a graph of terms whose edges are weighted by the semantic similarity between two terms (Vega-Oliveros et al. 2019). As this can not be adapted for multi-word catchphrase extraction this method is not applicable on our problem.

Out of the prior works stated above, some works (Vega-Oliveros et al. 2019; Nasar et al. 2019) focus on extracting single-word keywords and are not exactly applicable on our problem of multi-word catchphrase extraction. The rest of the methods (Frank et al. 1999; Liu et al. 2010; Hasan and Ng 2014; Siddiqi and Sharan 2015; Shi et al. 2017) which extract multi-word phrases as keyphrases, follow a classic two-phase extraction algorithm—(1) The first phase is to generate candidate terms eligible to be a keyphrase, and (2) The next phase is to prune out the keyphrases using some frequency-based numeric score. The supervised methods model the task as a binary classification problem where the model classifies a candidate phrase as either a keyphrase or not. In the present work, we have tried using regression models to prune out the catchphrases out of candidate phrases.

Catchphrase extraction from scholarly articles Catchphrase extraction from scholarly articles has been extensively studied since long and has gained recent interests. Among a few notable works, Caragea et al. developed both supervised and unsupervised methods for keyphrase extraction from scholarly documents (Caragea et al. 2014; Florescu and Caragea 2017). This work utilises the available citation-network among the articles to extract the catchphrases. Two recent works (Alzaidy et al. 2019; Zhu et al. 2020) on scholarly articles models the task of catchphrase extraction as a sequence labeling task and utilise the BiLSTM-CRF model. Alzaidy et al. directly uses the standard BiLSTM-CRF model and tests its performance on this task. Zhu et al. tests the performance while using

different pre-trained word embeddings along with the BiLSTM-CRF model. We consider the task of legal catchphrase extraction as a sequence labeling task, and use BiGRU-CRF model (a variation of BiLSTM-CRF model) to extract catchphrases from legal documents.

Catchphrase extraction from patents Patents are legal documents that are similar to court case statements in terms of grammatical and sentence structure. There have been attempts to extract catchphrases from *patents*. For instance, Suzuki et al. (Suzuki and Takatsuka 2016) used linguistic dependency structures to extract two types of keywords from patents—(1) representative keywords, that portray the subject category of the patent, and (2) novelty keywords, that bring out the inventiveness of the patent. Hu et al. (Hu et al. 2018) proposed a method called PKEA (Patent Keyword Extraction Algorithm) that extracts single-word keywords from patent documents that are helpful in categorising the patents into clusters. Okamoto et al. (Okamoto et al. 2017) used hand-crafted rules set by experts, to extract keyphrases (called mentions) from patent documents. They extracted two types of phrases, called '*technology mentions*' (important noun-phrases) and '*relation mentions*' (verbs connecting the technologies). Most of these methods make document-structure based assumptions specific to patent documents that are not applicable on court case documents. Also several of these methods only extract single-word keywords instead of multi-word catchphrases that are common in legal case documents.

Apart from the prior works of catchphrase extraction, there exists other works on patents where the role of important multi-word phrases was found to be helpful. Prior works (Zhou et al. 2014; Tannebaum and Rauber 2014) showed that important keyword-phrases/keyphrases of similar semantics can be used to expand queries for patent retrieval. The authors in (Tannebaum and Rauber 2014), showed that terms from the query can be converted to keyword-phrases to better represent the query. In the other work (Zhou et al. 2014), the authors showed that important noun phrases can potentially be considered as keyphrases for query expansion. Mahdabi et al. uses topic dependent important phrases for query expansion (Mahdabi and Crestani 2014). The topic dependent important multi-word phrases were extracted from topic dependent citation-graphs of patent documents. In another work (Al-Shboul and Myaeng 2014), the authors try to identify important multi-word phrases using the annotations in Wikipedia pages. These important phrases are used to enrich the query representations, which in turn results in better patent retrieval. The authors of one prior work (D'hondt et al. 2014) uses important multi-word phrases along with the traditional bag-of-words model to represent a document. This representation was found to be helpful in the task of patent categorization. All these works, demonstrate that important multi-word phrases can be helpful in many downstream information retrieval tasks such as: *patent document categorization* and *patent query expansion*.

Although, catchphrases are also important multi-word phrases, one vital aspect of a catchphrase is that they provide a concise representation of the document itself. The methods described above did not attempt to identify the document-specific importance of phrases, and therefore, these methods cannot be said to be exactly extracting catchphrases. In addition, these methods are unsupervised in nature. For these reasons, we have not considered these methods as baselines in the present work.

Survey papers on catchphrase extraction There have been several survey papers on comparing among methods for catchphrase extraction. Hasan and Ng compare and discuss several weaknesses in the keyphrase extraction methodologies and also present common challenges faced while extracting keyphrases (Hasan and Ng 2014). They specifically lay importance on a corpus-specific challenge that arises due to document size being larger relative to a fewer set of keyphrases per document. This difficulty is also present in our legal dataset, as case documents are lengthy and are annotated with only a few catchphrases per document. Another survey paper (Siddiqi and Sharan 2015) compares the performance of several supervised and unsupervised methods. It lists out commonly used features to prune the keyphrases out of all phrases within a document. Verberne et al. discusses five pre-existing methods for term extraction and compares their performance upon domain-specific text corpora (Verberne et al. 2016). They found that the most important factors that determine the success of a method, is the importance of a multi-word term (catchphrase) in the domain. Also, in their experiments, KL-divergence based method outperformed others. Although the methods were of unsupervised nature. (Nasar et al. 2019) is another survey of several catchphrase extraction techniques. It shows that several unsupervised attempts have been made to identify keywords from generic documents, but only a few supervised methods exist.

Tools for catchphrase extraction Apart from the research works, there also exist some tools for catchphrase extraction. For instance, KEA is a supervised tool for extracting keyphrases and ranking them (Witten et al. 1999). Another well-known supervised tool for catchphrase extraction is MAUI (Medelyan 2009) that is built over KEA. Both KEA and MAUI are used as baselines in this work, and are described in more detail in Sect. 5.2.

2.3 Other forms of information extraction from legal documents

There have been attempts towards other forms of information extraction from legal documents, that are similar to extraction of legal catchphrases. For instance, a closely related task to catchphrase extraction in legal documents is '*extraction of Indices*' (Le et al. 2015). Indices are crudely defined as chunks or clauses that contain as many legal terms as possible. Indices also include single-word keywords and multi-word phrases. However, the work (Le et al. 2015) is an unsupervised method meant for Japanese text and is hence not applicable to English documents.

Named entities are similar to *catchphrases* in that both are multi-word phrases obtained from the text. Although the property of catchphrases being document-specific does *not* hold true for named entities, and there are other distinct differences between the two, the task of recognizing *legal named entities* is comparable to extracting *legal catchphrases*, and has been implemented using Conditional Random Fields (CRF (Lafferty et al. 2001)) by Cardellino et al. (2017). CRF is a sequence tagger that when trained can identify meaningful parts out of a given sequence of data. In this paper, we have also used CRF to identify catchphrases (as a baseline). Various summarization techniques for legal documents including extractive catchphrase extraction have been discussed in (Bhattacharya et al. 2019). Recognizing

'*requisite*' and '*effectuation*' parts of a legal document is another form of information extraction from legal documents. Nguyen et al. (2018) and Truong et al. (2017) used a sequence tagger, specifically the BiLSTM-CRF model (Lample et al. 2016) for this task. In the present work, we use BiGRU-CRF model (a variation of the BiLSTM-CRF model) to extract catchphrases from legal documents.

3 Dataset for catchphrase identification

This section describes the dataset used for experiments in this paper. Various properties of the dataset are also explored, that brings out the challenges in the catchphrase identification problem.

The dataset consists of 400 court case reports of the Supreme Court of India, along with a set of catchphrases for each case report. The catchphrases are identified manually by the law practitioners associated with the *Manupatra* legal information system (https://www.manupatrafast.com/) which is a popular system among law practitioners in India, and can be accessed via a paid subscription. The Manupatra catchphrases corresponding to three case reports are shown in Table 2 (second column). Note that the Manupatra website secures its contents via protocols that forbid any form of automated download. Hence, each case report and its catchphrases were collected through manual browsing. This laborious manual process prevented us from collecting a larger number of documents and their catchphrases.

Although we were unable to create a larger dataset, it can be noted that standard catchphrase extraction datasets have used similar number of documents. For instance, the dataset released in SemEval-2017 Keyphrase extraction task (Augenstein et al. 2017) had 500 scholarly articles among which 350 were for training.[5] Again, the ACL-keyphrase corpus consists of 500 scientific paragraphs.[6] These two datasets were used by (Zhu et al. 2020) to train a BiLSTM-CRF model for keyphrase extraction from scholarly articles. Hence, we can consider our dataset of 400 court case statements to be adequate in size for training a sequence tagging model.

In our dataset, 94.9% of the catchphrases associated with a particular case report are present within the text of the corresponding document, on average. Hence, this dataset is suitable for extractive catchphrase detection from within the text of the document (which is preferred by the law practitioners in India).

3.1 Properties of the dataset

We now present some statistics of our dataset (summarized in Table 3), which also help to understand the challenges associated with the identification of catchphrases from the case documents.

[5] https://scienceie.github.io/index.html.

[6] https://nlp.stanford.edu/pubs/FTDDataset_v1.txt.

Table 3 Statistics of our dataset of Indian Supreme Court case documents and the gold standard catchphrases

Number of case reports	400
Number of sentences	55,259 (approx.)
Number of Catchphrases	9032
Average number of catchphrases per document	22.5 (9032/400)
Number of catchphrases present within the document text	8570 (94.9% of 9032)
Correlation between number of catchphrases and document length	0.25
Average number of occurrence of a catchphrase in a document	1.85
Number of catchphrases that are noun phrases	88% (approx.)

• The documents are very long with the 100 longest documents having more than 10,000 words each on average. However, the number of catchphrases is as low as 22.5 per document (see Table 3).

- The frequency of occurrence of catchphrases in a document is very low—the average frequency of occurrence of a catchphrase in a document is just 1.85 times. Moreover, the number of (distinct) catchphrases per document varies hugely. The standard deviation of the number of catchphrases per document is 24.35. Also, the number of catchphrases is *not* correlated to the length of the document. The correlation between the number of words in the document and number of catchphrases is as low as 0.25.
- The length of individual catchphrases (measured as the number of words) varies widely, from a minimum of one word (unigram) to as long as seven words. So, traditional chunking of n-grams renders an inefficient solution. Examples of lengthy catchphrases include '*Central Board of Excise and Customs*', '*Board for Industrial and Financial Reconstruction*', '*Arbitrators by the Chief Justice of India*', '*Freedom of Conscience and Free Profession*', and so on. Also note that many catchphrases contain stop-words in between, that forbids the stop-word removal process done traditionally in most applications.

All these statistics suggest that catchphrases are very rarely occurring phrases in long documents, and highlight the challenges in identifying the catchphrases. Also it can be noted that a huge majority (88%) of the catchphrases in the dataset are in the form of *noun phrases*, which is evident from the examples given in Table 2 (second column). This insight can be utilized for their detection, as explained in the later sections of the paper.

3.2 Train-test split for supervised methods

As stated earlier, this paper develops supervised methodologies for catchphrase identification. We adopt a 10-fold cross validation approach for training and evaluating all supervised methods discussed in this work. In other words, we partition the dataset into 10 partitions of 40 documents each. In each fold, we use one of the

partitions as the test set, and the rest nine partitions (360 documents) for training. Finally, the performance measures are averaged over all the ten folds.

3.3 A collection of unlabelled documents for learning embeddings

In addition to the labelled set of 400 documents, we have a separate collection of unlabelled documents containing 33, 545 legal case documents from the Supreme Court of India. This collection includes an exhaustive list of court cases from the year 1950 to 2016, and are collected from a legal archive (http://liiofindia.org/in/cases/cen/INSC/). Note that this legal archive does *not* provide catchphrases for the case documents. This unlabelled set of documents has been used in this work to learn document embeddings by training the Doc2Vec model (Le and Mikolov 2014)) (as is described in Sect. 4.3).

To ensure fairness of evaluation, it has been ensured that documents in this unlabelled collection do *not* have any overlap with the labelled set of 400 documents.

4 Methods for catchphrase identification

This section describes various supervised methods for catchphrase extraction from legal case documents. We first describe the classic and intuitive approach of learning to score candidate phrases, and explain why this approach is unsuitable. Next we present an approach of modelling the catchphrase extraction task as a sequence labelling problem. Finally, we describe our proposed approach (which is an improvement over the second approach) consisting of a novel neural-based sequence labeling model for supervised catchphrase extraction.

4.1 Approach 1: identifying catchphrases by scoring candidate phrases

Before the advent of neural models for catchphrase extraction, most methods (Frank et al. 1999; Witten et al. 1999; Tomokiyo and Hurst 2003; Medelyan 2009; Liu et al. 2010; Hasan and Ng 2014; Siddiqi and Sharan 2015; Shi et al. 2017) traditionally followed a two-step solution—(1) identifying candidate phrases from the text, and (2) scoring these candidate phrases and choosing a certain number of top-scoring ones as the final catchphrases. Among the prior works on legal-specific catchphrase extraction, the two unsupervised methods (Galgani et al. 2012; Mandal et al. 2017) also follow this two-step approach. In this approach, we follow the same two steps towards the task of catchphrase extraction, as described below.

4.1.1 Identifying candidate phrases

While selecting the set of candidate phrases for a document, it is desirable to select a smaller set of candidate phrases that includes as many catchphrases as possible. Different prior works have adopted different approaches to identify candidate phrases. For instance, Galgani et al. (Galgani et al. 2012) considers sentences, while

Table 4 Examples of noun phrases (NNPs) extracted by our customized noun phrase extractor and by SpaCy (https://spacy.io)

Text 1	The Supreme Court has reiterated that, even in a case of **death by a single blow**, but on the vital part of the body, the case may fall under Section 302 of the Indian Penal Code (Murder).
SpaCy NNPs 1	*The supreme court, a case, death, a single blow, the vital part, the body, the case, section, the indian penal code, murder*
Our NNPs 1	*Blow, body, case, case of death, case of death by single blow, code, court, death,* ***death by a single blow****, indian penal code, murder, part, penal, penal code, section, section of the indian penal code, single blow, supreme court, vital part, vital part of the body*
Text 2	Whenever a plea of **omission** to put a question to the accused on **vital piece of evidence** is raised in the appellate court, courses available to the appellate court can be briefly summarised as under
SpaCy NNPs 2	*A plea,* ***omission****, a question, vital piece, evidence, the appellate court, courses, the appellate court*
Our NNPs 2	*Appellate court, appellate court courses, courses, court, court courses, evidence,* ***omission****, piece, plea, plea of omission, question, vital piece,* ***vital piece of evidence***

The actual catchphrases in all fields are highlighted in boldface. Our customized NNP_extractor can extract many more catchphrases than SpaCy

others (Tomokiyo and Hurst 2003) considers *n*-grams, while our prior work (Mandal et al. 2017) considers noun phrases as candidate phrases. Using sentences as candidate phrases is *not* suitable in our case, since we wish to get exact match with gold standard catchphrases. Hence our choice was limited to noun phrases or *n*-grams (for some suitable value of *n*).

Considering noun phrases as candidate phrases We decided to consider *noun phrases* as the candidate phrases, due to several reasons. First, several law practitioners (from West Bengal National University of Juridical Sciences, and Rajiv Gandhi School of Intellectual Property Law) with whom we discussed affirmed that legal catchphrases are mostly noun phrases. This view is confirmed from our dataset that has 88% of its catchphrases as noun phrases (see examples in Table 2). Prior works (Gollapalli et al. 2017; Hasan and Ng 2014; Vega-Oliveros et al. 2019; Nasar et al. 2019) also tried to use noun phrases as candidate catchphrases. Additionally, our experiments showed that using noun phrases gives better overall performance. For all these reasons, we decided to consider noun phrases as candidate phrases. A brief comparison between considering *n*-grams and considering noun phrases is presented later in Sect. 5.2.3 that will justify our decision of using noun phrases as candidate phrases.

A customized extractor for noun phrases from legal documents There exist several popular Natural Language Processing tools that can be directly used to extract noun phrases, e.g., SpaCy (https://spacy.io). However, we observed that the noun phrases identified by these existing tools are *not* fully suitable for our purpose. Tools such as SpaCy only identify *non-intersecting* phrases, and the set of noun phrases identified may miss many potential catchphrases (as shown through an example in Table 4). Hence we designed a customized noun phase extractor specifically for our purpose. We have made the implementation of our customized noun phrase

extractor available online[7]. Table 4 shows some examples to demonstrate that our NNP_extractor can extract many noun phrases that are actually catchphrases, but are missed by SpaCy.

4.1.2 Scoring the candidate phrases

We apply regression models to score the candidate phrases in a supervised setting. Regressors are similar to binary classifiers except that, instead of producing a discrete value of either zero or one, the regressors output a continuous value in the range [0, 1], and this value can be interpreted as the probability of a candidate phrase being a catchphrase. We used different supervised regressors to score the set of candidate phrases extracted from a document, such as: Linear Regressor,[8] Logistic Regressor,[9] SVM Regressor (Hearst et al. 1998), Decision Tree Regressor (Breiman et al. 1983), Random Forest Regressor (Breiman 2001), and MLP Regressor (Hinton 1990). We use the implementations for all these regression models that are available in the Python *scikit-learn* library (https://scikit-learn.org/stable/).

To train the regression models, *fourteen hand-crafted features* are extracted from each candidate phrase. These features include (1) *Phrase-specific features*, e.g. length of the phrase, number of words in the phrase, (2) *Document-specific features*, e.g. number of occurrence of the phrase in the document, position of the first occurrence of the phrase in the document, and (3) *Domain-specific features*, e.g. number of occurrence of the phrase in the corpus, number of legal terms present in the phrase. For a given candidate phrase in the training set, a label 0 is assigned if it is not a catchphrase and label 1 is assigned if it is a catchphrase.

The trained regression models are then used to predict the scores of candidate phrases extracted from a document in the test set. The candidate phrases are sorted in decreasing order of the value output by the trained regressor, and a certain number of the top-scoring candidate phrases are considered to be the predicted catchphrases for the said document.

We find that this *approach of using supervised regressors for identifying catchphrases achieves very poor results*. Out of the regressors stated above, the Random Forest Regressor performs the best—the top-scoring 133 catchphrases extracted by Random Forest Regressor achieves the best F1-Score (out of all regressors) of 0.026—but this performance is much poorer than that of the other models discussed in the later sections. Hence, detailed results of this approach are omitted for brevity. We believe that the poor performance of the regressors is because they are not using any contextual information of the phrases, but such contextual information is necessary for inferring catchphrases. Hence, in our next approach, we model the catchphrase identification task as a sequence labelling task where the sentence-level contextual information of words will be used.

[7] The GitHub url to our noun phrase extractor is https://github.com/amarnamarpan/NNP-extractor.
[8] https://en.wikipedia.org/wiki/Linear_regression.
[9] https://en.wikipedia.org/wiki/Logistic_regression.

Table 5 Sample of input data format for the sequence taggers, shown for the text snippet "*The main controversy in the Civil Appeal was, whether on acceptance of any bipartite settlement between the management ...*" that contains the three catchphrases '*controversy*', '*civil appeal*', and '*bipartite settlement*'

Word	POS	Label
The	DT	O
Main	JJ	O
Controversy	NN	B-catch
In	IN	O
The	DT	O
Civil	JJ	B-catch
Appeal	NN	I-catch
Was	VBD	O
,	,	O
Whether	IN	O
On	IN	O
Acceptance	NN	O
Of	IN	O
Any	DT	O
Bipartite	JJ	B-catch
Settlement	NN	I-catch
Between	IN	O
The	DT	O
Management	NN	O

4.2 Approach 2: modelling the task as a sequence labelling task

In the recent years, sequence labelling models (also called sequence taggers) have gained popularity for tasks such as *Named Entity Recognition* (NER) (Lample et al. 2016; Chiu and Nichols 2016). A sequence tagger is fundamentally a classifier, whose input is presented as a sequence of tokens (e.g., words) along with their features and labels, and the task is to assign a class/label for each token in the sequence.

We apply sequence taggers for the identification of catchphrases as follows. The document text is first broken into sentences. Then, each sentence is tokenized into a sequence of words. Then for each word, we extract some features (e.g., Parts-of-Speech tags) and append it to the corresponding words. Also for each word, we assign one of three labels *B-catch*, *I-catch*, and *O*. The label *B-catch* denotes a word that is the beginning of a catchphrase, the label *I-catch* denotes a word that is inside a catchphrase, and the label *O* is used for other words (that are not part of any catchphrase). A demonstration of the labels used while training is provided in Table 5 for an example text snippet. During prediction, the input format is the same, except that the labels are not provided.

Note that, in this work, the sequence tagging models are applied in a sentence-by-sentence manner. We also tried applying the sequence tagging models over the whole document at once, but that approach gave us much inferior evaluation results. Hence we are only reporting the results for sequence tagging models that are applied over one sentence at a time.

We have experimented with two sequence taggers. We briefly describe them below.

4.2.1 Conditional random fields (CRF)

CRF or Conditional Random Fields (Lafferty et al. 2001) is a sequence labelling method that is used for various NLP tasks including part-of-speech tagging and named entity recognition. We use two features for training the CRF to infer the label for words—(1) the case-folded word, and (2) the part-of-speech (POS) tag of the word. The first feature is simply the word itself, where all the alphabets have been converted to lowercase. The part-of-speech tag of the word is a tag that describes the grammatical form of the word[10] (ex: verb, noun, adjective, conjunctive etc.). Note that each training data point is a sentence tokenized into a sequence of words along with their POS tags.

For all the experiments, we use the implementation of CRF in pyCRFsuite[11]. For training the CRF model, we use the *L-BFGS* algorithm for 100 iterations and the coefficients $L1$ and $L2$ are set to their default values 1.0 and 0.001 respectively.

An auxiliary benefit of using CRF is that it can provide *confidence scores* for each of the catchphrases that it extracts. CRF uses the viterbi algorithm (Lafferty et al. 2001) for determining tag-wise probabilities. This internal mechanism for scoring each tag is called the *viterbi accuracy*[12]. These viterbi accuracy scores can be aggregated by taking a mean to get the final score of the catchphrase, and this score can be considered to be an estimate of the importance of individual catchphrases. In situations where we need to restrict the number of catchphrases extracted from a document, we can use this score to retain just the top-scored K catchphrases (for some suitable value of K) instead of the whole set of extracted catchphrases.

A major drawback of CRF is that if a word has not been encountered during training, CRF cannot consider the word to be a part of any new catchphrase. To overcome this limitation, we employ another sequence tagging model which takes into account the character-level structure and the sequence of surrounding words to infer the characteristics of the new unseen word.

4.2.2 BiGRU-CRF

We use the sequence tagging model called BiGRU-CRF that is a minor variation of the BiLSTM-CRF model described in (Lample et al. 2016). The variation of this model is obtained by simply replacing the LSTM cells by GRU cells. Actually, we considered both BiLSTM-CRF and BiGRU-CRF in our experiments, and observed better performance in case of BiGRU-CRF. We suspect that this improved performance for BiGRU-CRF is due to the fact that GRU cells have lesser number

[10] The complete list of POS tags can be found at www.ling.upenn.edu/courses/Fall_2003/ling001/penn_treebank_pos.

[11] Available at https://pypi.org/project/python-crfsuite/.

[12] To get viterbi accuracy scores in pyCRFsuite one can use the '*-i*' option while tagging.

of parameters to be tuned during the training phase, which enables the training of BiGRU-CRF to converge faster on our relatively small training set (360 documents).

Similar to the BiLSTM-CRF model in (Lample et al. 2016), the BiGRU-CRF model has the following four layers

Layer 1 Two recurrent cells each decoding the word level embedding (of 100 dimensions) and character level embedding (of 25 dimensions). Each of the recurrent cells is a Bidirectional-GRU unit;

Layer 2 A Bidirectional-GRU layer (of 100 dimensions);

Layer 3 A single layer of fully connected neural network (of 150 dimensions);

Layer 4 A CRF layer (Lafferty et al. 2001) of 3 dimensions corresponding to three output labels.

The BiGRU-CRF model is trained for 25 epochs with a batch size of 15, using Adam optimizer, having learning rate of 0.001, ($\beta_1 = 0.9$ and $\beta_2 = 0.99$). Also, each training data point is just a sentence tokenized into a sequence of words. Further details of the architecture can be found in (Lample et al. 2016). The results for the sequence taggers are presented later in Sect. 7.

4.3 Approach 3: using document context information with sequence taggers

As discussed in the previous section, sequence taggers can be used to identify catchphrases just as they can be used to identify named entities. However, there is a fundamental difference between a word-sequence that is a named entity and a word-sequence that is a legal catchphrase. We discuss this difference, and our proposed approach that is motivated by this difference between named entities and legal catchphrases.

4.3.1 Difference between identifying named entities and identifying catchphrases

Named entities can be identified by learning the general layout of a named entity, e.g., the structural form of the word-sequence, the structure of the words (and their POS tags) within the word-sequence, and the dependency between the predicted tags themselves. Importantly, whether a word-sequence is a named entity does *not* usually depend on the textual content of the document in which the word-sequence occurs; a word-sequence that is a named entity in one document is also likely to be a named entity in any other document.

Unlike named entities, catchphrases are document-specific word-sequences that give some form of summarization of the document content. So, whether a particular word-sequence is a catchphrase in a given document depends on the complete text and context of that document, and *not* on just the context of the sentence that is passed at a time to the sequence tagger for prediction. In fact, a word-sequence that is considered to be a catchphrase in one document is unlikely to be considered as a catchphrase in another document if the contexts of the two documents are dissimilar. For example, if a civil case is related to *human rights*, then a possible catchphrase would be 'fundamental rights'; whereas, if a criminal case is related to *homicide*, the same sequence of words 'fundamental rights' is unlikely to be selected as

a catchphrase. Whereas, if a word-sequence is a named entity in one document, it is likely to be a named entity in all other documents, regardless of their contexts. Hence, identification of catchphrases needs one more important ingredient to be learnt by the model, which is the relation of a candidate word-sequence with the overall text of the document. This aspect is *not* important in the traditional named entity recognition task, but is crucial for the catchphrase extraction task. Hence, for the model to infer a relation between the document text and the predicted tags, we need to *pass some representation of the document to the model* at a suitable stage. This observation motivates our third (proposed) approach that is described next.

4.3.2 Adding document representation to the BiGRU-CRF model

In this approach, we provide a document representation to the BiGRU-CRF model (discussed earlier in Sect. 4.2.2) so that, instead of just getting trained by the form of the catchphrases, the model can learn the relation between the context of the catchphrases and the context of the corresponding documents.

To get a representation of a document, we use the popular Doc2Vec model (Le and Mikolov 2014) which is an unsupervised technique of estimating document vectors. The underlying model is a neural network used to predict a document vector along with the word vectors when given a document. The word vectors formed are shared across documents (see (Le and Mikolov 2014) for details). For training the Doc2Vec model, we used a separate dataset of 33, 545 case documents from the Indian Supreme Court (this dataset is described in Sect. 3.3). We used the *gensim* implementation of Doc2Vec[13] with the default hyperparameter values. We used a 100-dimensional Doc2Vec vector for representing the document context.

The architecture of our proposed model, which we call D2V-BiGRU-CRF, is shown in Fig. 1. The bottom-most layer is a bidirectional language model (Peters et al. 2018) that extracts word and character embeddings from the word sequence using a deep network of its own. Character embeddings of size 25 and word embeddings of size 100 are concatenated together and then fed to the Bidirectional GRU layer (the second layer). The Doc2Vec embedding of the document is concatenated with the outputs of the second layer (the bidirectional GRU layer), which is then fed into the fully connected layer. The fully connected layer (with a dropout rate of 0.5) learns an enveloped abstraction (a weight matrix) over the combined word vectors and the document context vector. The output of this fully connected layer further connects to a CRF layer which performs the final three-class prediction (B-catch, I-catch, and O). As an added benefit, this model has CRF as its last layer. Hence, it can assign confidence scores to the extracted catchphrases (viterbi accuracy scores, as stated in Sect. 4.2.1).

The proposed D2V-BiGRU-CRF model is developed using the *Keras*[14] library in Python. The implementation of the developed model is available at https://github.com/

[13] available at http://radimrehurek.com/gensim/index.html.

[14] available online at https://keras.io/.

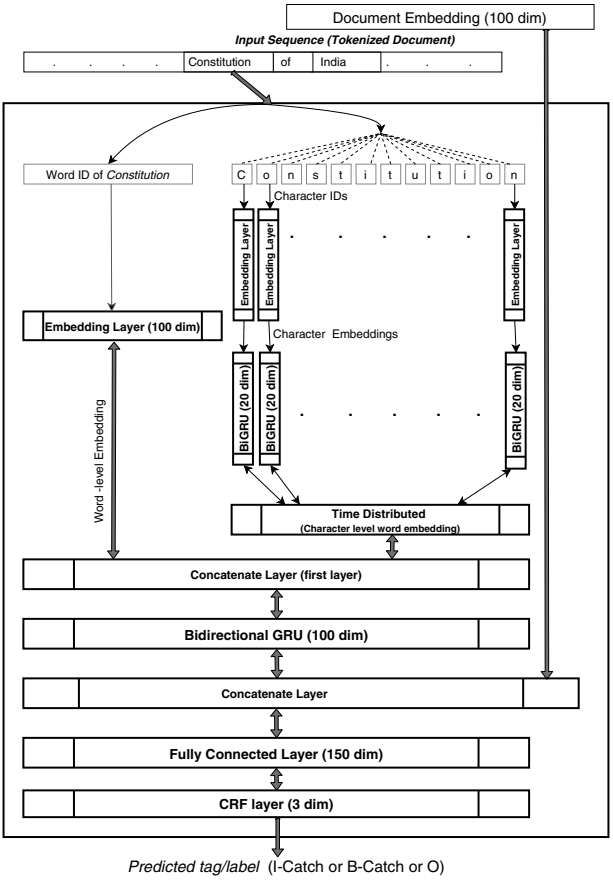


Fig. 1 The layer architecture of our proposed model for catchphrase extraction. The model is built on Keras, and has three inputs—the word ID, the character IDs, and the Document Context Vector. There is only one output giving us the label ID

amarnamarpan/D2V-BiGRU-CRF. We will see in Sect. 7 how addition of the document context vector enables significantly better identification of catchphrases.

It can be noted that our proposed architecture can also be used with other types of document representations. We specifically chose to use Doc2vec since it was found in a recent work (Mandal et al. 2021) on estimating legal document similarity that Doc2vec embeddings gave the best performance among many forms of document representations (even better than BERT-based embeddings). We leave it as future work to try out different kinds of document representations for the task of extracting catchphrases.

5 Baseline methods for catchphrase extraction

This section describes some existing methods for catchphrase extraction (both unsupervised and supervised) that have been considered as baselines in our work.

5.1 Unsupervised baselines for catchphrase extraction

We chose as baselines two unsupervised methods developed specifically for catchphrase extraction from legal case documents—*PSlegal* (Mandal et al. 2017) and *MyScore* (Galgani et al. 2012). Both these methods are essentially word/phrase scoring functions, that are used to score candidate phrases. We consider as candidate phrases, the noun phrases extracted by our noun phrase extractor (described in Sect. 4.1.1).

5.1.1 PSlegal

This is a scoring function that uses frequency based statistical measures to rank the candidate phrases (Mandal et al. 2017). To check the likelihood of a candidate phrase c to be a catchphrase in a particular document d, the following factors are taken into account—(1) specificity of the candidate phrase c in the document, (2) the domain-specific importance of c in a legal corpus, and (3) presence of legal terms within c. The final score is a product of the above three terms.

The first factor, specificity of a candidate phrase in the document, is computed using *KLI* (Tomokiyo and Hurst 2003) which is a KL-divergence based informativeness measure:

$$KLI(c,d) = \frac{TF(c,d)}{|d|} \cdot \log \frac{(\frac{TF(c,d)}{|d|})}{(\frac{CF(c)}{n})}$$

where, $CF(c)$ is the total number of occurrences of the candidate phrase c in the whole corpus, $TF(c, d)$ is the number of occurrences of c in the document d, n is the total number of documents in the corpus, and $|d|$ denotes the length of the document in words.

The second factor measures the domain-specific importance (DSI) of c, by comparing the importance of the phrase in a legal corpus and a non-legal corpus, as

$$DSI(c, \mathbb{C}_l, \mathbb{C}_{nl}) = \frac{Imp(c, \mathbb{C}_l)}{Imp(c, \mathbb{C}_{nl}) + 1}$$

where $\mathbb{C}_l$ denotes the given legal corpus and $\mathbb{C}_{nl}$ denotes a non-legal corpus (specifically, a large collection of general news articles was considered in (Mandal et al. 2017)), and the importance of c in any corpus $\mathbb{C}$ is measured as $\frac{CF(c,\mathbb{C})}{DF(c,\mathbb{C})+1}$, where $CF(c, \mathbb{C})$ is the total number of times that c occurs in the corpus $\mathbb{C}$, and $DF(c, \mathbb{C})$ is the number of documents in $\mathbb{C}$ where c is present.

Table 6 Top ten ranked candidate phrases from the same document with and without the three different factors mentioned for PSlegal

With factor 1 only	With factors 1 and 2	PSlegal (with factors 1, 2 and 3)
object and reason	Consumer	Consumer
Writ	Forum	Consumer dispute
Mean	Consumer forum	Court
Contempt	Redressal	Forum
Majority	Act	Commission
Thereunder	Reason of the provision	Civil court
Level	Power	Act
Price	Section	Redressal
Statutory authority	Trade pratice	President
Industry	Summary trial	Right of the consumer

For the third factor, those candidate phrases that contain a legal term/phrase within it, are given a higher weight than those which do not. To this end, a large dictionary $\mathbb{D}$ of legal terms/phrases was compiled from various online sources. Subsequently, for a candidate phrase c (considered an ordered set of words), it was checked how well the set of words in c matches with any legal phrase $p \in \mathbb{D}$. Only those matches were considered where a legal phrase $p \in \mathbb{D}$ is fully contained within c.

Table 6 shows examples of candidate catchphrases identified using with factor (1) only, with factors (1) and (2), and considering all the three factors described above. The final score of a candidate phrase is computed as the product of the three weights stated above. All candidate phrases (e.g., all noun phrases) are ranked with respect to this score, and a certain number of the top-scoring candidate phrases are extracted as catchphrases. The reader is referred to (Mandal et al. 2017) for details.

5.1.2 MyScore

MyScore, developed by Galgani et al. (Galgani et al. 2012), considers a candidate phrase to be a set (bag) of terms. Three scores are computed for each term, and these scores are combined to compute the score for a candidate phrase.

For a particular term, three frequency-based scores are computed—TF, TF-IDF, and *NOccur*. The TF and TF-IDF scores are well-known, hence are not described for brevity. The new measure *NOccur* is the frequency of occurrence of a term in the 'important sentences' of the document. Given a document, the sentences are scored on some basis (described hereafter), and the top-scoring sentences are considered as the important sentences. The sentences are scored as follows. For a particular sentence, a discrete set of the words in the sentence is considered, and the average term-frequencies of each of these words (in the given document that contains the sentence) is computed. Those sentences that have the highest *mean term frequency* are chosen as the important sentences.

After calculating the TF-IDF, TF and *NOccur* for each term in the document, the individual scores are combined as follows. Three ranked lists of the terms are created using the three different scores. The final rank of a term is the mean of its ranks in the three ranked lists. The final score of a term is computed based on this final rank of the term.

Finally, the score of a candidate phrase (considered as a set of its constituent terms) is computed as the mean (average) of the scores of all its constituent terms. The candidate phrases with the highest scores thus computed are identified as the catchphrases. Since the implementation of this method is not publicly available, the method was implemented by us following the description in (Galgani et al. 2012).

5.2 Supervised baselines for catchphrase extraction

We consider as baselines three supervised methods of catchphrase extraction—KEA (Witten et al. 1999), MAUI (Medelyan 2009), and DeepCNN (Tran et al. 2020, 2018). KEA and MAUI are generic catchphrase extraction tools, while DeepCNN is specific to the legal domain. We describe them briefly in the rest of this section.

5.2.1 KEA

KEA (http://www.nzdl.org/Kea/) is a machine learning based tool that, given some training examples of documents with their catchphrases, learns a model for extraction of catchphrases from new documents. The training starts with creation of candidate phrases (considered to be *n*-grams of pre-defined lengths, default 1 to 5 words). A set of features is extracted for each candidate phrase, including TF-IDF score, the position of first occurrence, and so on. The probability distributions of these features of the catchphrases (in the training data) are compared with the distributions of those candidate phrases which are not catchphrases. This contrast in probability distributions form the trained model, and is used during extraction of catchphrases from new documents. The functioning of KEA is detailed in (Witten et al. 1999).

5.2.2 MAUI

MAUI (https://github.com/HIIT/maui-2) is essentially a topic indexing model that can be used for catchphrase extraction or topic extraction. Similar to KEA, MAUI is also a supervised method which requires training examples of documents and their corresponding catchphrases. It is built upon four open-source components—KEA (Witten et al. 1999) for n-gram keyphrase extraction, Weka[15] for topic indexing, Jena that integrates extrinsic vocabularies, and Wikipedia Miner.[16] The details of MAUI can be found in (Medelyan 2009).

[15] Available at https://www.cs.waikato.ac.nz/ml/weka/.

[16] github.com/dnmilne/wikipediaminer.

5.2.3 DeepCNN

DeepCNN (Tran et al. 2018, 2020) is a recent model based on Convolutional Neural Networks, that is designed specifically for catchphrase extraction from legal documents. We obtained an implementation of DeepCNN upon request from the authors of (Tran et al. 2018, 2020). The method has two steps:

Step 1 Given a document, the CNN model is used to assign a score to each word, that depicts how likely a word is to be part of a catchphrase. Subsequently, a threshold number of the highest-scored words are identified as the *anchor words*.

Step 2 Then, each anchor word is considered along with a pre-defined number of previous words on its left and subsequent words on its right as a catchphrase.

In their original works (Tran et al. 2018, 2020), Tran et al. chose ten anchor words per document, and considered five previous and five subsequent words for each anchor word in the middle, as a catchphrase of length eleven words. Hence, the original DeepCNN model can only identify a fixed number of catchphrases per document (equal to the number of anchor words identified), and each catchphrase is of a fixed odd length (e.g., 11 words). However, the catchphrases in our dataset are of variable lengths, and the original DeepCNN method is not suitable for our dataset. Hence we applied different variations as described below.

Out of the two steps of the DeepCNN method (described above), Step 1 is kept as-is, i.e., a score is assigned to each word, and a definite number of the highest-scored words are selected as the anchor words. The second step is where the following two adaptations are made, so that the method can give catchphrases of variable lengths.

Variation 1: n-gram form—After the anchor words are obtained, we consider all n-grams (for some defined value of n) that enclose an anchor word to get the final catchphrases. We considered two versions: (1) **DeepCNN-2**, where we consider all n-grams with $n \leq 2$ (i.e., unigrams and bigrams) surrounding each anchor word, and (2) **DeepCNN-3**, where we consider all n-grams with $n \leq 3$ (i.e., unigrams, bigrams and trigrams) surrounding each anchor word as a catchphrase. Please note, for each anchor word we can have one unigram, two bigrams, three trigrams and likewise n number of n-grams. Larger ranges of n gives us a much larger number of catchphrases thereby reducing the performance in terms of precision. For brevity, we omit the results using larger ranges of n as they provide inferior results.

Variation 2: noun phrase form—Here we identify the noun phrases (if any) that enclose each anchor word. Subsequently, these noun phrases (that contain one or more anchor words) are considered to be the final catchphrases. If an anchor word is not enclosed by a noun phrase, it is considered as-is (without any neighboring word) as the final catchphrase. We call this variation **DeepCNN-NNP**.

Also, the number of anchor words chosen for the documents in our dataset is best suited to be 50 (instead of 10 in the original DeepCNN model). This number was experimentally found to give the best F1-Score as compared to using 100, 75, 30, 20 or 10 anchor words per document. Thus, for all the three variations stated above, we considered the number of anchor words to be identified as 50.

6 Evaluation

The methods discussed in this paper are of various types. Some methods are supervised, while others are unsupervised. Also, some of the methods output a fixed set of catchphrases, while the other methods output a ranked list of phrases. This section describes how we evaluate all these methods in a common setting.

6.1 Evaluating supervised and unsupervised methods

The unsupervised methods (*PSlegal* and *MyScore*) do not require any training phase. Hence, they are directly applied to each of the 400 documents in our dataset.

On the other hand, for evaluation of the supervised methods (KEA, MAUI, DeepCNN, the Sequence Taggers, and our proposed model D2V-BiGRU-CRF), we perform a **ten-fold cross validation** over the set of 400 documents. The ten-fold cross validation allows the prediction model to be run upon each of the 400 documents exactly once. And renders better out-of-sample accuracy. It means that catchphrases for every single document is predicted in such a way that no document in the prediction set is used to train the model.

Thus, each unsupervised method and each supervised method is applied exactly once over each of the 400 documents, and the average performance over all documents is reported for each method.

6.2 Evaluating methods that output a fixed set of catchphrases or a ranked list of catchphrases

The methods discussed in this paper have two types of output—some of the methods output a fixed set of catchphrases, while others output a ranked list of phrases such that the top-ranked ones are more likely to be catchphrases. We intend to apply set-based evaluation metrics (Precision, Recall, F1-score, as described later in this section) for comparing the various methods. The methods that output a fixed set of catchphrases (all sequence taggers, the DeepCNN variants, and our proposed model) can be evaluated directly using the set-based measures.

But set-based metrics are not directly applicable for methods that give a ranked list of phrases as output (PSlegal, MyScore, KEA and MAUI). As stated earlier, these methods use different scoring techniques to assign scores to words/phrases, where the highly scored ones are the most suitable ones to be catchphrases. The user is expected to choose a certain number of top-ranked words/phrases as the catchphrases. Hence, to ensure a fair comparison of these scoring methods using set-based measures, we have to choose a certain number n of the top-scored phrases from the complete ranked list. For each method, we experiment with various values of n in the range [10, 400], and we choose that value of n which results in the highest F1-score. The cutoff n for each method are as follows—MyScore: 203, PSlegal: 104, KEA: 333, MAUI: 43.

Thus, the output of every method is now a set of extracted catchphrases. We can now apply set-based evaluation metrics (Precision, Recall and F1-Score) for evaluating their performances.

6.3 Evaluation metrics

For every document in our dataset, we have a gold-standard set of catchphrases obtained from the Manupatra legal system (see Sect. 3), and catchphrases identified by a particular method. In order to calculate any metric of evaluation (how well a method has identified catchphrases from a document), we need to first identify the true positives when given a set of gold-standard catchphrases and a set of extracted catchphrases (both sets for the same document).

To this end, we consider two possible ways of measuring true positives—(i) *exact match* and (ii) *rouge match*. An *exact match* renders two phrases to be a match when there is a word-by-word verbatim match between the two phrases. Whereas, *rouge match* (Lin 2004) is a more relaxed form where a partial match can also be considered as a match. This form of evaluation was used in the baseline works by Galgani et al. (2012). Here, two phrases (one from the gold-standard set and the other from the extracted set) is considered to match if the *Rouge recall score*[17] between the two phrases is higher than 0.5.

After we compute the number of true positives using either of the above two methods we now use the following metrics to evaluate the output (set of extracted catchphrases) of every method:

- *Precision* the fraction of the extracted catchphrases that match with a gold-standard catchphrase; calculated as $Precision = \frac{TruePositives}{TruePositives+FalsePositives}$.
- *Recall* the fraction of the gold-standard catchphrases that are correctly extracted as a catchphrase, i.e., match with an extracted catchphrase; calculated as $Recall = \frac{TruePositives}{TruePositives+FalseNegatives}$.
- *F1-Score* the harmonic mean of Precision and Recall.

7 Results

This section compares the performances of the various catchphrase detection methods, in terms of the metrics stated in the previous section.

7.1 Comparative analysis of different methods

Table 7 compares the performance of all methods with respect to *exact match* and Table 8 compares the same with respect to *rouge match*. Both tables are divided into

[17] To compute rouge recall score we use the implementation found at https://pypi.org/project/rouge-score/.

Table 7 Comparing performances of all catchphrase extraction methods using *EXACT match*

Metrics/methods	Recall	Precision	F1-Score	N_c
Unsupervised Methods with ranked list of Catchphrases				
MyScore (Galgani et al. 2012)	0.181	0.022	0.050	(203)
PSlegal (Mandal et al. 2017)	0.417	0.094	0.163	(104)
Supervised Methods with ranked list of Catchphrases				
KEA (Witten et al. 1999)	0.287	0.037	0.075	(333)
MAUI (Medelyan 2009)	0.164	0.091	0.127	(43)
Supervised Methods with fixed set of Catchphrases				
DeepCNN-3 (variation of (Tran et al. 2018))	0.421	0.069	0.118	101.92
DeepCNN-2 (variation of (Tran et al. 2018))	0.414	0.077	0.130	91.10
DeepCNN-NNP (variation of (Tran et al. 2018))	0.339	0.162	0.219	33.22
CRF (Lafferty et al. 2001)	0.213	0.489	0.297	10.60
BiGRU-CRF based on (Lample et al. 2016)	0.399	0.532	0.456	18.22
D2V-BiGRU-CRF (proposed)	**0.527**	**0.638**	**0.577**	19.40

The proposed method outperforms all other methods (best values in bold font). The last column N_c, shows the average number of catchphrases identified by a method; the values within parentheses are chosen (top n from a ranked list) so as to maximize the F1-score

Table 8 Comparing performances of all catchphrase extraction methods using *ROUGE match*

Metrics/Methods	Recall_{rg}	Precision_{rg}	F1-Score_{rg}
Unsupervised Methods with ranked list of Catchphrases			
MyScore (Galgani et al. 2012)	0.770	0.082	0.148
PSlegal (Mandal et al. 2017)	**1.000**	0.221	0.362
Supervised Methods with ranked list of Catchphrases			
KEA (Witten et al. 1999)	**1.000**	0.120	0.215
MAUI (Medelyan 2009)	**1.000**	0.150	0.260
Supervised Methods with fixed set of Catchphrases			
DeepCNN-3 (Variation of (Tran et al. 2018))	**1.000**	0.189	0.318
DeepCNN-2 (Variation of (Tran et al. 2018))	0.489	0.089	0.151
DeepCNN-NNP (Variation of (Tran et al. 2018))	0.665	0.532	0.591
CRF (Lafferty et al. 2001)	0.339	0.746	0.466
BiGRU-CRF based on (Lample et al. 2016)	0.555	0.729	0.630
D2V-BiGRU-CRF (proposed)	0.799	**0.947**	**0.866**

The proposed method outperforms all other methods (best values in bold font). The subscript *rg* in the column headings signify that the true positives used to compute precision, recall and F1-Score were computed using rouge match

three blocks. The first block lists the performances of the unsupervised methods; the last column states the respective cut-offs (n values) in parentheses, where the top-ranked n phrases are considered as catchphrases (as detailed in the previous section). Similarly, the second block shows the supervised tools KEA and MAUI that also

give ranked lists, and the top-ranked n (stated in the last column inside parentheses) are considered as catchphrases. The third block lists the performance of various versions of the DeepCNN model, the sequence taggers, along with our proposed approach D2V-BiGRU-CRF. These methods output a set of catchphrases, and the last column shows the average number of catchphrases identified per document.

The obtained results show that our proposed model outperforms all other methods in terms of all metrics Recall, Precision and F1-Score for both exact match and rouge match. In fact, the performance of the proposed method is *statistically significantly better than that of all baselines in all the evaluation measures* at 95% confidence level ($p < 0.05$) by Wilcoxon signed-rank test (Siegel 1956). Table 2 (third column) shows a few examples of catchphrases extracted by the proposed model.

We have the following additional observations from the results:

- Generic methods / tools of catchphrase extraction (*KEA, MAUI*) do not perform well in extracting catchphrases from legal documents. Though these methods achieve very high Recall for rouge match, their precision values are very low thus leading to low F-score (in both Table 7 and Table 8).
- Among the variations of DeepCNN, the variation using noun-phrases (DeepCNN-NNP) outperforms the two variations using n-grams (DeepCNN-2 and DeepCNN-3). The n-gram versions identify a much larger number of catchphrases (average number of catchphrases being above 90) than DeepCNN-NNP (average number of catchphrases being 33.2). As a result, the n-gram versions have higher Recall, but much lower Precision compared to DeepCNN-NNP, thus resulting in much higher overall performance (F1-score) for DeepCNN-NNP. This observation shows the advantage of using noun phrases over using n-grams for legal catchphrase extraction, and justifies the decision stated in Sect. 4.1.
- The proposed D2V-BiGRU-CRF performs much better than the BiGRU-CRF model across all measures. This comparison shows that including document context drastically improves performance of the catchphrase extraction model.

7.2 A qualitative analysis of catchphrases

We finally provide a qualitative analysis of the task of catchphrase extraction and the performance of our proposed sequence tagging model. We primarily target two questions: (1) What kind of catchphrases are easier or harder to detect? (2) What type of catchphrases is our proposed model uniquely good at detecting?

Which catchphrases are easier/harder to detect? To estimate whether a particular catchphrase is easier or harder to detect, we check how many of the different methods discussed earlier are successful in extracting the said catchphrase. The intuition is that, the catchphrases that are harder to detect can be extracted by a fewer number (or none) of the methods, and those catchphrases that are easier to detect can be detected by most (if not all) of the methods.

Table 9 shows a few of the catchphrases that are easiest to detect (i.e., which could be extracted by almost all the methods) and also a few that are harder to detect (i.e., which could be extracted by only one of the methods). We also show a few

Table 9 Examples of gold standard catchphrases that are easier to detect (could be detected by all methods discussed in this paper) along with those that are much harder to detect (could be detected by only one method, or by none at all)

Detected by all methods (Easier to detect)	Detected by only one method (Harder to detect)	Not detected by any method (Hardest to detect)
Welfare	Waiver of right to object	Manufacture of final product
Lease	Contract to the contrary	Air quality standard
Guarantee	Private service vehicle	Non application of mind
Refund	View of the apology	Matters of public employment
Penalty	Insurance against third party risk	Vexatious claim
Gratification	Distribution of legislative power	Public account
Natural justice	Application for setting aside	Building and engineering contract
Factory	Enforcement of fundamental right	Nil rate of duty
Amendment	Offences against public	Act done in good faith
Defamation	Restrictive trade practice	Cinema show
Tribunal	Procedure established by law	Karbate tube

examples of catchphrases that could not be detected by any of the methods discussed earlier. We observe that, the catchphrases that are more difficult to detect are consistently *longer* than the ones that are easier to detect. Also many of the catchphrases that could not be detected at all do *not* contain terms of general legal significance, rather these phrases seem to be important in the specific context of the corresponding case.

Which catchphrases is our proposed method uniquely good at detecting? To answer this question, we check which of the catchphrases could be correctly identified by our proposed method, but could *not* be extracted by any of the other methods. Some examples of such catchphrases that are solely detected by the proposed method (and not by any other method) include: *extension of time*, *difference of opinion*, *appointing authority*, *period of limitation*, *corruptly making report*, *right of appeal*, *rate of duty*, *burden of proof*, *rate of duty*, *condition precedent*, *breach of contract*, *screening an offender*. We note that almost all of these catchphrases are of multiple words. Also, many of these catchphrases do *not* include any individual legal keyword, but the word sequence as a whole has a legal significance.

8 Discussion

The current research on catchphrase/keyphrase extraction (Zhu et al. 2020; Alzaidy et al. 2019) have made use of neural sequence labeling models that has been previously shown to be the state-of-the-art in the task of Named Entity Recognition (Lample et al. 2016). The main argument proposed in this paper is that *"catchphrases are document-specific phrases and hence sequence labeling models should be able to use document-specific information to extract more accurate catchphrases"*. Our work tries to distinguish between a named entity and a catchphrase, where a named

entity is *not* dependent on the document-context but a catchphrase is. Our experiments show that a simple intuitive modification in an existing sequence labeling model gives us a novel sequence tagging model (D2V-BiGRU-CRF) that can utilise the document-context information. Our model extracts catchphrases that are specific to the given document, rather than extracting phrases that would have been considered catchphrases if a different document was provided. This distinction was impossible for the model to make without the provided document-specific information.

Our experiments show that our proposed model D2V-BiGRU-CRF significantly outperforms both state-of-the-art unsupervised and supervised methods for legal catchphrase extraction. As with any other sequence labeling model, the main strength of our proposed model is that it does not require any hand-crafted features and is hence fully automatic. To demonstrate the results of our model, we have provided the catchphrases extracted by our model for three example case documents from the Indian Supreme Court in Table 2. Our contributions are as follows:

- The task of catchphrase extraction from legal documents is modeled as a sequence labelling problem, so that existing sequence labeling models become applicable to this task. We found that this approach gives promising results, achieving higher Precision and Recall (both of which are important for a legal information system) even when identifying a smaller set of catchphrases than other methods.
- We proposed a novel supervised method for extraction of legal catchphrases, by adding a document representation to a sequence labeling model. Specifically, we added a Doc2Vec embedding to a specific layer of a sequence labeling model that uses Bidirectional GRU blocks as its core. This addition of the document vector enables the model to better extract catchphrases specific to a given document.

It can be noted that the task of catchphrase extraction is a *subjective* task (as opposed to an objective task for which the gold standard is rigidly defined), and differences occur even when different domain experts are asked to extract catchphrases from the same document. For instance, it was shown in (Galgani et al. 2012) that, when human experts were asked to extract catchphrases from court case documents, their identified catchphrases reached a maximum *F1-Score* of only 0.53 when compared to the catchphrases suggested by the original authors of those documents. Given this scenario, any model is unlikely to achieve 100% F1-Score with respect to a given set of gold standard catchphrases. Hence, the F1-Score of 0.57 achieved by our proposed approach can be considered as a relatively high score.

The catchphrases identified by our model have several practical applications. For instance, they can be used to index legal case documents in legal information systems, which will help efficient search. They can be used for document categorization as they precisely represent a concise summary of a document. Again, the catchphrases indicate the key legal issues in a case document, and hence can be useful for identifying similar cases (which is technically known as the 'similar case matching' problem (Zhong et al. 2020)). We plan to study the applicability of the detected catchphrases for such tasks in future.

In future, we plan to explore whether the developed algorithm can be used for extracting catchphrases from legal documents of other domains, e.g., those of other countries. Also, while in this work we only experimented with Doc2vec document embeddings, we plan to explore how varying the type of document-embedding affects the quality of the catchphrases. Finally, the set of catchphrases presents a form of summary of a given document. Although in this work we only explore *extractive* approaches of catchphrase detection, we might improve the set of catchphrases by incorporating *abstractive* approaches. A potential future direction of work would be to explore the applicability of *abstractive* methods along with a more robust form of evaluation that includes manual evaluation by experts.

Acknowledgements The authors acknowledge faculty members from The West Bengal National University of Juridical Sciences (www.nujs.edu), and Rajiv Gandhi School of Intellectual Property Law (www.iitkgp.ac.in/department/IP) for insightful discussions. The research is partially supported by the TCG Centres for Research and Education in Science and Technology (CREST) through the project titled 'Smart Legal Consultant: AI-based Legal Analytics'. The first author is supported by the Visvesvaraya PhD scheme from the Ministry of Electronics and Information Technology (Grant No. VISPHDMEITY-1570).

Funding The first author received his research grants from the "Ministry of Electronics and Information Technology, Government of India" via granting the fellowship "Visvesvaraya PhD Scheme for Electronics and IT". The research is partially supported by the TCG Centres for Research and Education in Science and Technology (CREST) through the project titled 'Smart Legal Consultant: AI-based Legal Analytics'.

Code availability The implementation of the proposed model (D2V-BiGRU-CRF) is available at https://github.com/amarnamarpan/D2V-BiGRU-CRF.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

Al-Shboul B, Myaeng SH (2014) Wikipedia-based query phrase expansion in patent class search. Inform Retrieval J 17:430–451

Alzaidy R, Caragea C, Giles CL (2019) Bi-lstm-crf sequence labeling for keyphrase extraction from scholarly documents. In: Proceedings of the International Conference on World Wide Web, pp 2551–2557

Augenstein I, Das M, Riedel S, Vikraman L, McCallum A (2017) SemEval 2017 task 10: ScienceIE - extracting keyphrases and relations from scientific publications. In: Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017), pp 546–555

Bhattacharya P, Hiware K, Rajgaria S, Pochhi N, Ghosh K, Ghosh S (2019) A comparative study of summarization algorithms applied to legal case judgments. In: Advances in Information Retrieval, pp 413–428

Bizer C, Lehmann J, Kobilarov G, Auer S, Becker C, Cyganiak R, Hellmann S (2009) Dbpedia-a crystallization point for the web of data. J Web Semantics 7(3):154–165

Breiman L (2001) Random forests. Mach learn 45(1):5–32

Breiman L, Friedman JH, Olshen RA, Stone CJ (1983) Classification and regression trees. CRC Press, Cambridge

Caragea C, Bulgarov FA, Godea A, Das Gollapalli S (2014) Citation-enhanced keyphrase extraction from research papers: A supervised approach. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, pp 1435–1446

Cardellino C, Teruel M, Alemany LA, Villata S (2017) A low-cost, high-coverage legal named entity recognizer, classifier and linker. In: Proceedings of International Conference on Articial Intelligence and Law), pp 9–18

Chiu JP, Nichols E (2016) Named entity recognition with bidirectional LSTM-CNNs. Trans Assoc Comput Linguist 4:357–370

Dhondt E, Verberne S, Oostdijk N, Beney J, Koster C, Boves L (2014) Dealing with temporal variation in patent categorization. Inform Retrieval J 17:520–544

Firoozeh N, Nazarenko A, Alizon F, Daille B (2019) Keyword extraction: issues and methods. Nat Lang Eng 26:259–291

Florescu C, Caragea C (2017) PositionRank: An unsupervised approach to keyphrase extraction from scholarly documents. In: Proceedings of Annual Meeting of the Association for Computational Linguistics, pp 1105–1115

Frank E, et al. (1999) Domain-specific keyphrase extraction. In: International Joint Conference on Artificial Intelligence, pp 668–673

Galgani F, et al. (2012) Towards automatic generation of catchphrases for legal case reports. In: Proceedings of Computational Linguistics and Intelligent Text Processing (CICLing), pp 414–425

Giamblanco N, Siddavaatam P (2017) Keyword and Keyphrase Extraction using Newton's Law of Universal Gravitation. Proceedings of Canadian Conference on Electrical and Computer Engineering pp 1–4

Gollapalli SD, Li X, Yang P (2017) Incorporating expert knowledge into keyphrase extraction. In: Association for the Advancement of Artificial Intelligence

Hasan KS, Ng V (2014) Automatic keyphrase extraction: A survey of the state of the art. In: Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp 1262–1273

Haveliwala TH (2002) Topic-sensitive pagerank. In: Proceedings of the International Conference on World Wide Web, p 517–526

Hearst MA, Dumais ST, Osuna E, Platt J, Scholkopf B (1998) Support vector machines. IEEE Intell Syst Appl 13(4):18–28

Hinton GE (1990) Connectionist learning procedures. In: Machine Learning, pp 555 – 610

Hu J, Li S, Yao Y, Yu L, Yang G, Hu J (2018) Patent keyword extraction algorithm based on distributed representation for patent classification. Entropy 20(2):104

Lafferty JD, McCallum A, Pereira FCN (2001) Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In: Proceedings of International Conference on Machine Learning, pp 282–289

Lample G, Ballesteros M, Subramanian S, Kawakami K, Dyer C (2016) Neural architectures for named entity recognition. In: Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp 260–270

Le Q, Mikolov T (2014) Distributed representations of sentences and documents. In: Proceedings of International Conference on Machine Learning, pp 1188–1196

Le TTN, Shirai K, Nguyen ML, Shimazu A (2015) Extracting indices from Japanese legal documents. Art Intell Law 23(4):315–344

Lin CY (2004) ROUGE: A package for automatic evaluation of summaries. In: Text Summarization Branches Out, Association for Computational Linguistics, Barcelona, Spain, pp 74–81, https://www.aclweb.org/anthology/W04-1013

Liu Z, Li P, Zheng Y, Sun M (2009) Clustering to find exemplar terms for keyphrase extraction. In: Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing, p 257–266

Liu Z, Huang W, Zheng Y, Sun M (2010) Automatic keyphrase extraction via topic decomposition. In: Proceedings of the 2010 conference on Empirical Methods in Natural Language Processing, pp 366–376

Lossio-Ventura JA, Jonquet C, Roche M, Teisseire M (2016) Biomedical term extraction: overview and a new methodology. Inform Ret J 19:59–99

Mahdabi P, Crestani F (2014) The effect of citation analysis on query expansion for patent retrieval. Inform Ret J 17:412–429

Mandal A, Ghosh K, Pal A, Ghosh S (2017) Automatic catchphrase identification from legal court case documents. In: Conference on Information and Knowledge Management, ACM, New York, USA, CIKM '17, pp 2187–2190

Mandal A, Ghosh K, Ghosh S, Mandal S (2021) Unsupervised approaches for measuring textual similarity between legal court case reports. Artificial Intelligence and Law

Medelyan O (2009) Human-competitive automatic topic indexing. PhD thesis, The University of Waikato, New Zealand
Nasar Z, Jaffry SW, Malik MK (2019) Textual keyword extraction and summarization: state-of-the-art. Inform Process Manag 56(6):102088
Nguyen S, Nguyen LM, Tojo S, Satoh K, Shimazu A (2018) Recurrent neural network-based models for recognizing requisite and effectuation parts in legal texts. Artificial Intelligence and Law pp 1–31
Okamoto M, Shan Z, Orihara R (2017) Applying information extraction for patent structure analysis. In: Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, p 989–992
Peters M, Neumann M, Iyyer M, Gardner M, Clark C, Lee K, Zettlemoyer L (2018) Deep contextualized word representations. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), pp 2227–2237
Qazvinian V, Radev DR, Özgür A (2010) Citation summarization through keyphrase extraction. In: Proceedings of Conference on Computational Linguistics, pp 895–903
Shi W, Zheng W, Yu JX, Cheng H, Zou L (2017) Keyphrase extraction using knowledge graphs. Data Sci Eng 2(4):275–288
Siddiqi S, Sharan A (2015) Keyword and keyphrase extraction techniques: a literature review. Int J Comput Appl 109(2)
Siegel S (1956) Nonparametric Statistics for the Behavioral Sciences. McGraw-Hill series in psychology, McGraw-Hill
Suzuki S, Takatsuka H (2016) Extraction of keywords of novelties from patent claims. In: Proceedings of Conference on Computational Linguistics, pp 1192–1200
Tannebaum W, Rauber A (2014) Using query logs of uspto patent examiners for automatic query expansion in patent searching. Inform Ret J 17:452–470
Tomokiyo T, Hurst M (2003) A language model approach to keyphrase extraction. In: Proceedings of ACL Workshop on Multiword Expressions: Analysis, Acquisition and Treatment, pp 33–40
Tran V, Le Nguyen M, Tojo S, Satoh K (2020) Encoded summarization: summarizing documents into continuous vector space for legal case retrieval. Artificial Intelligence and Law pp 1–27
Tran VD, Nguyen ML, Satoh K (2018) Automatic catchphrase extraction from legal case documents via scoring using deep neural networks. CoRR arxiv:abs/1809.05219
Truong S, Le Minh N, Satoh K, Satoshi T, Shimazu A (2017) Single and multiple layer bi-lstmcrf for recognizing requisite and effectuation parts in legal texts. In: Proceedings of Automated Semantic Analysis of Information in Legal Texts
Vega-Oliveros DA, Gomes PS, Milios EE, Berton L (2019) A multi-centrality index for graph-based keyword extraction. Inform Process Manag 56(6):102063
Verberne S, Sappelli M, Hiemstra D, Kraaij W (2016) Evaluation and analysis of term scoring methods for term extraction. Inform Ret J 19(5):510–545
Witten IH, Paynter GW, Frank E, Gutwin C, Nevill-Manning CG (1999) Kea: Practical automatic keyphrase extraction. In: Proceedings of the Fourth ACM Conference on Digital Libraries, p 254–255
Wu YFB, Li Q (2008) Document keyphrases as subject metadata: Incorporating document key concepts in search results. Inform Ret J 11:229–249
Zahoor F, Bajwa IS (2014) Automatic extraction of catchphrases from software license agreement. Proceedings of International Conference on Intelligent Human-Machine Systems and Cybernetics 2:189–193
Zhong H, Xiao C, Tu C, Zhang T, Liu Z, Sun M (2020) How does NLP benefit legal system: A summary of legal artificial intelligence. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp 5218–5230
Zhou D, Truran M, Liu J, Zhang S (2014) Using multiple query representations in patent prior-art search. Inform Ret J 17:471–491
Zhu X, Lyu C, Ji D, Liao H, Li F (2020) Deep neural model with self-training for scientific keyphrase extraction. Public Library of Science (Plos one) 15(5):e0232547

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Arpan Mandal[1] · Kripabandhu Ghosh[2] · Saptarshi Ghosh[3] · Sekhar Mandal[1]

Kripabandhu Ghosh
kripa.ghosh@gmail.com

Saptarshi Ghosh
saptarshi@cse.iitkgp.ac.in

Sekhar Mandal
sekhar@cs.iiests.ac.in

[1] Department of Computer Science and Technology, Indian Institute of Engineering Science and Technology Shibpur, Howrah, India

[2] Department of Computational and Data Sciences (CDS), Indian Institute of Science Education and Research (IISER) Kolkata, Kolkata, West Bengal, India

[3] Department of Computer Science and Engineering, Indian Institute of Technology Kharagpur, Kharagpur, West Bengal, India

View publication stats